Reporte

Nombre: Osmar Abdi Sanchez Santiago Omar Tadeo Gasca Deciga Gabriela Alejandra Fernández Briones	**Matrícula:** 2226699 2226708 2226870
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor:** Miguel Ángel Gómez Marroquín
Módulo Netacad: Unidad 5	**Actividad:** ACT05-U5 Laboratorio guiado
Fecha: 05/11/2026	

Bibliografía (Formato APA)

Beck, K. (2002). Test-driven development: By example. Addison-Wesley.

Python Software Foundation. (s. f.). unittest.mock — mock object library. https://docs.python.org/3/library/unittest.mock.html

pytest-dev. (s. f.). pytest documentation. https://docs.pytest.org

SQLAlchemy. (s. f.). SQLAlchemy documentation. https://docs.sqlalchemy.org

GitHub. (s. f.). GitHub Actions documentation. https://docs.github.com/actions

Objetivo:

Comprobar con pruebas automatizadas que las reglas de un sistema de pedidos se cumplen: un pedido VIP recibe 15 % de descuento; no se confirma con importe cero o negativo ni con deuda del cliente; el cobro se simula con una pasarela ficticia; los pedidos se guardan en SQLite en memoria sin tocar datos reales y existen reglas de cancelación. Después se transfiere lo aprendido a tres reglas de Eventos Nova.

Procedimiento:

Resultados de aprendizaje

- Distinguir pruebas unitarias, de integración y extremo a extremo (E2E).
- Aplicar Arrange–Act–Assert y el ciclo TDD Red–Green–Refactor.
- Verificar persistencia y rollback de SQLAlchemy sin modificar datos de producción.
- Simular cobros con mocks, probar autorización y revisar cobertura.
- Interpretar un pipeline CI y entregar evidencia reproducible.

Tipos de prueba

TIPO	QUÉ VERIFICA	EJEMPLO EN ESTE LABORATORIO
Unitaria	Una regla aislada, sin base de datos ni servicios externos.	Order.confirm() con VIP de 200 devuelve 170.
Integración	Varios componentes juntos (código + SQLAlchemy + SQLite).	Guardar un pedido, commit y consultarlo; rollback y no encontrarlo.
E2E	Un recorrido completo desde la entrada hasta el resultado.	Solo propuesto (sección 11): el proyecto base no tiene API ni interfaz.

Estado de las evidencias

Se conservan las imágenes originales del alumno (E1, RED, GREEN, cobertura, htmlcov, las tres ejecuciones de GitHub Actions y la ejecución de las pruebas del PIA). Solo falta la captura de Ruff en E7.

ID	REQUISITO DE LA GUÍA	EN ESTE REPORTE	PENDIENTE DEL ALUMNO
E1	Entorno y salida de pytest	Comandos y captura original	Ninguno
E2	Pruebas 200 VIP y 200 regular	Código, tabla y captura original (Fig. 1)	Ninguno
E3	TDD Red–Green–Refactor	Código y capturas RED y GREEN originales	Ninguno
E4	Commit, rollback, id=2	Tabla y explicación; PASSED en Fig. 1	Código de la prueba
E5	Mock, rechazo VIP 200 → 170	Código, riesgo y PASSED en Fig. 1	Ninguno
E6	Autorización y límite de 24 h	Matriz	Captura PASSED de seguridad
E7	Cobertura ≥ 80 % y Ruff	Comandos y capturas originales	Captura de Ruff
E8	CI verde → rojo → verde	URL y 3 capturas originales de Actions	Ninguno
E9	≥ 9 casos del PIA	Casos, código, CSV y captura original (Fig. 11)	Ninguno

Preparación del entorno (E1)

COMPONENTE	DESCRIPCIÓN
Lenguaje	Python 3.11 o superior
Pruebas	pytest y pytest-cov
Persistencia	SQLAlchemy con SQLite en memoria
Simulación externa	unittest.mock.Mock (no hay cobros reales)
Calidad	Ruff y GitHub Actions
Proyecto base	05_Proyecto_Base_U5.zip (src/, tests/, ci.yml, requirements.txt)

WINDOWS (POWERSHELL)	MACOS / LINUX
py -3.11 -m venv .venv · .\.venv\Scripts\Activate.ps1	python3 -m venv .venv · source .venv/bin/activate
python -m pip install -r requirements.txt	python -m pip install -r requirements.txt
python -m pytest -v	python -m pytest -v

venv aísla las bibliotecas del curso de otros proyectos. pip instala pytest, SQLAlchemy, Ruff y pytest-cov. pytest descubre las funciones cuyo nombre empieza por test_ y reporta PASSED o FAILED.

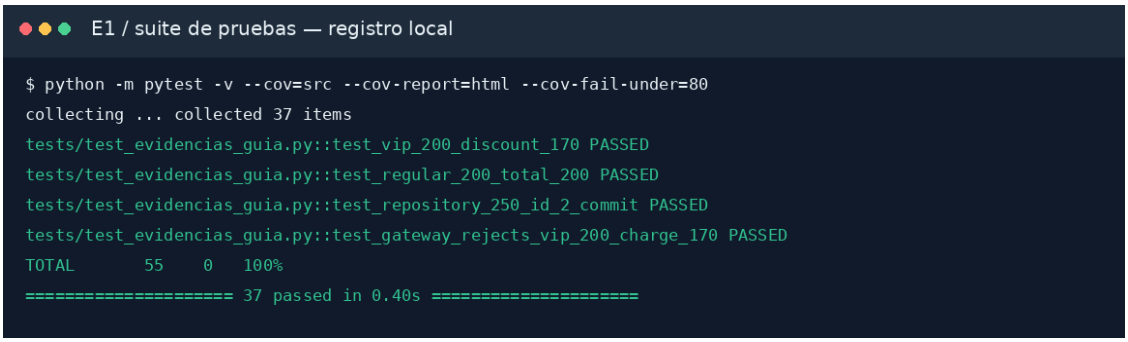


Figura 1. Evidencia E1: suite de pruebas y cobertura (imagen original del alumno).

Pruebas unitarias y Arrange–Act–Assert (E2)

Order.confirm() rechaza totales ≤ 0, rechaza deuda pendiente, aplica 15 % a clientes VIP y cambia el estado a CONFIRMED. No usa base de datos, por eso es una prueba unitaria.

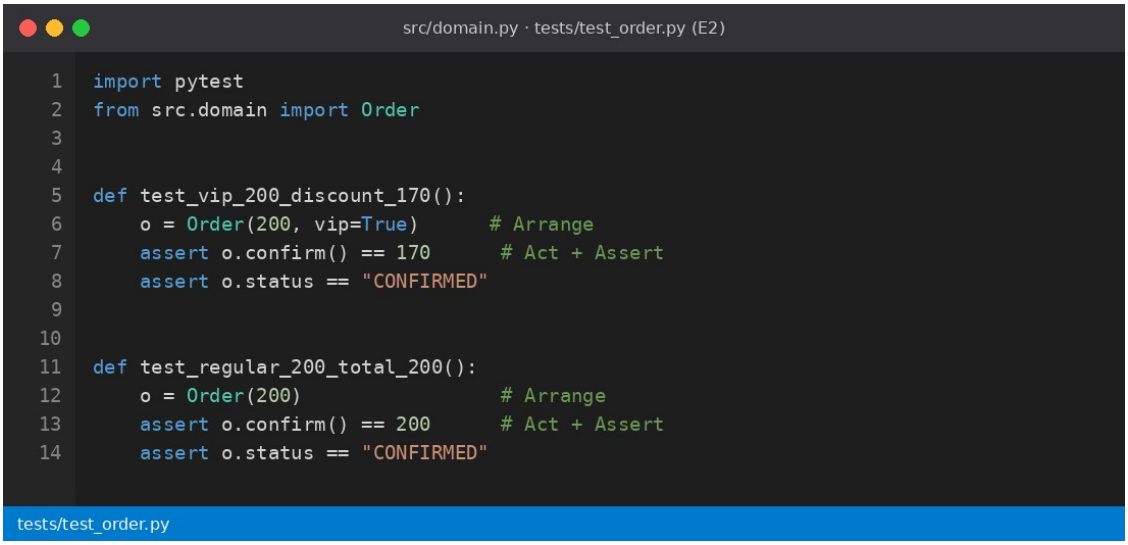


Figura 2. Dos pruebas nuevas de E2 (Arrange, Act y Assert).

ENTRADA	REGLA	RESULTADO ESPERADO	RESULTADO REAL
Total=200; VIP	Descuento 15 %	Cobro 170; CONFIRMED	170; CONFIRMED
Total=200; regular	Sin descuento	Cobro 200; CONFIRMED	200; CONFIRMED
Total=100; deuda	Deuda bloquea	CustomerHasDebt	CustomerHasDebt
Total=0 o -1	Importe positivo	ValueError	ValueError

TDD: Red → Green → Refactor (E3)

La regla del proyecto ya valida totales positivos, por lo que repetirla no produciría la fase RED. Se trabaja una regla nueva: no confirmar un pedido ya confirmado.

RED: primero la prueba

TDD · fase RED (tests/test_order.py)

```python
def test_cannot_confirm_twice():
    order = Order(100)
    order.confirm()

    with pytest.raises(ValueError, match="already confirmed"):
        order.confirm()

# Ejecutar ANTES de modificar src/domain.py  ->  debe FALLAR (RED)
```

tests/test_order.py

Figura 3. Prueba escrita antes de implementar la regla.

E3 / RED — reproducción sobre código base

```
$ python -m pytest tests/test_tdd_red.py -v
tests/test_tdd_red.py::test_cannot_confirm_twice FAILED [100%]
> with pytest.raises(ValueError, match="already confirmed"):
E Failed: DID NOT RAISE <class 'ValueError'>
==================== 1 failed in 0.05s ====================
```

Figura 4. Captura RED: la prueba falla porque el código base permite confirmar dos veces (imagen original).

GREEN: implementación mínima

TDD · fase GREEN (src/domain.py)

```python
class Order:
    ...
    def confirm(self):
        if self.status == "CONFIRMED":
            raise ValueError("already confirmed")
        # ...validaciones de total, deuda y descuento VIP...
        self.status = "CONFIRMED"
        return amount
```

src/domain.py

Figura 5. Implementación mínima en src/domain.py.

E3 / GREEN — versión corregida

```
$ python -m pytest tests/test_csv_cases.py::test_second_confirmation_rejected_after_first_confi…
tests/test_csv_cases.py::test_second_confirmation_rejected_after_first_confirmation PASSED
==================== 1 passed in 0.03s ====================
```

Figura 6. Captura GREEN: la prueba nueva pasa tras la corrección (imagen original).

REFACTOR

Se mejora el diseño sin cambiar el comportamiento, por ejemplo con una excepción de dominio específica (OrderAlreadyConfirmed) o un método privado de validación, y se vuelve a ejecutar toda la suite.

TDD no es ejecutar pruebas después de programar: la prueba se escribe primero, se observa que falla (lo cual demuestra que la prueba detecta el defecto) y solo entonces se implementa lo mínimo para pasarla.

Persistencia y transacciones (E4)

Aquí interviene SQLAlchemy con una base SQLite en memoria; cada prueba crea su propio motor, así que no se modifica ninguna base real.

PRUEBA	OPERACIÓN	RESULTADO ESPERADO
Commit	Guardar pedido id=1 total 120 y session.commit()	La consulta por id=1 devuelve total 120
Rollback	Guardar y session.rollback()	La consulta devuelve None
Actividad	Guardar pedido id=2 total 250, commit	id=2, total=250 y estado esperado del modelo

Commit vs. rollback: commit hace permanentes los cambios de la transacción en la base de prueba; rollback los descarta y deja la base como estaba. SQLite en memoria no sustituye todas las pruebas de un motor productivo como PostgreSQL.

Código de la prueba de la actividad: tests/test_repository.py (adaptar a los nombres de tu SqlOrderRepository y OrderRow). Captura de PASSED: pegar en E4.

Servicios externos con mocks (E5)

Un mock es un objeto controlado por la prueba que responde como el servicio externo. No se transmite dinero.

tests/test_service.py · mock de pasarela (E5)

```python
from unittest.mock import Mock
import pytest
from src.domain import Order
from src.application import OrderService


def test_gateway_rejects_vip_200_charge_170():
    gateway = Mock()
    gateway.charge.return_value = False
    order = Order(200, vip=True)

    with pytest.raises(RuntimeError, match="payment rejected"):
        OrderService(gateway).checkout(order)

    gateway.charge.assert_called_once_with(170)
```

tests/test_service.py

Figura 7. Rechazo de pago de un pedido VIP de 200: se intenta cobrar 170 y se lanza RuntimeError.

ESCENARIO	COMPORTAMIENTO ESPERADO	RESULTADO
Aprobado, total 100	charge(100)=True; devuelve 100	PASSED
Rechazado, VIP 200	charge(170)=False; RuntimeError	PASSED
Cargo real	Ninguno; el gateway es Mock	Sin pagos reales

Riesgo de consistencia: el código base confirma el pedido antes de conocer la respuesta de la pasarela; si el pago se rechaza, el pedido queda CONFIRMED sin cobro.

Propuesta de mejora: autorizar el pago primero y confirmar o persistir el pedido solo con respuesta aprobada, dentro de una transacción controlada, con idempotencia y reintentos seguros.

Seguridad y autorización (E6)

Prioridad implementada en can_cancel: un pedido SHIPPED nunca se cancela, ni siquiera por un administrador; en los demás estados el administrador puede cancelar; el propietario no administrador solo puede cancelar durante las primeras 24 horas. Las pruebas usan fechas fijas y pasan now de forma explícita.

ROL / IDENTIDAD	ESTADO	ANTIGÜEDAD	DECISIÓN	PRUEBA
Administrador, ajeno	NEW	30 días	Permitido	Actividad E6
Administrador, propietario	SHIPPED	0 horas	Denegado	Existente
Propietario, no admin	NEW	23 horas	Permitido	Existente
Propietario, no admin	NEW	24 h exactas	Denegado	Actividad E6
Propietario, no admin	NEW	25 horas	Denegado	Existente
Tercero, no admin	NEW	0 horas	Denegado	Existente

Cobertura y calidad (E7)

Cobertura del 80 % significa que al menos 80 % de las líneas de src/ se ejecutaron durante las pruebas; no significa que se detectó 80 % de los defectos.

COMANDOS
python -m ruff check src tests
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-fail-under=80
Abrir htmlcov/index.html en el navegador



Figura 8. Evidencia E7: resumen de cobertura en consola, 100 % (imagen original).

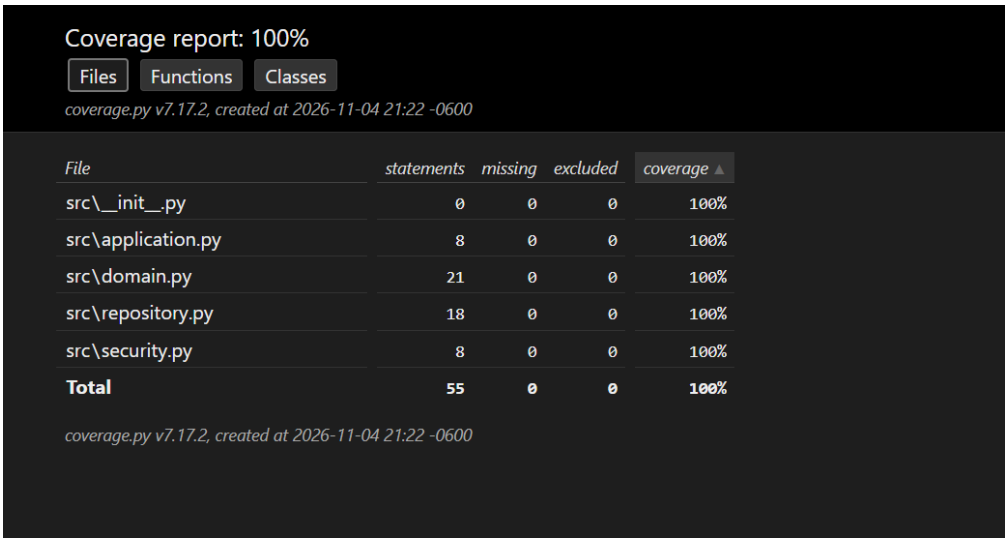


Figura 8b. Reporte HTML de cobertura abierto en el navegador (imagen original).

Rama no probada: escribir aquí una rama de src/ que aún no se ejecute (ver columna Missing) y por qué es relevante probarla.

Integración continua con GitHub Actions (E8)

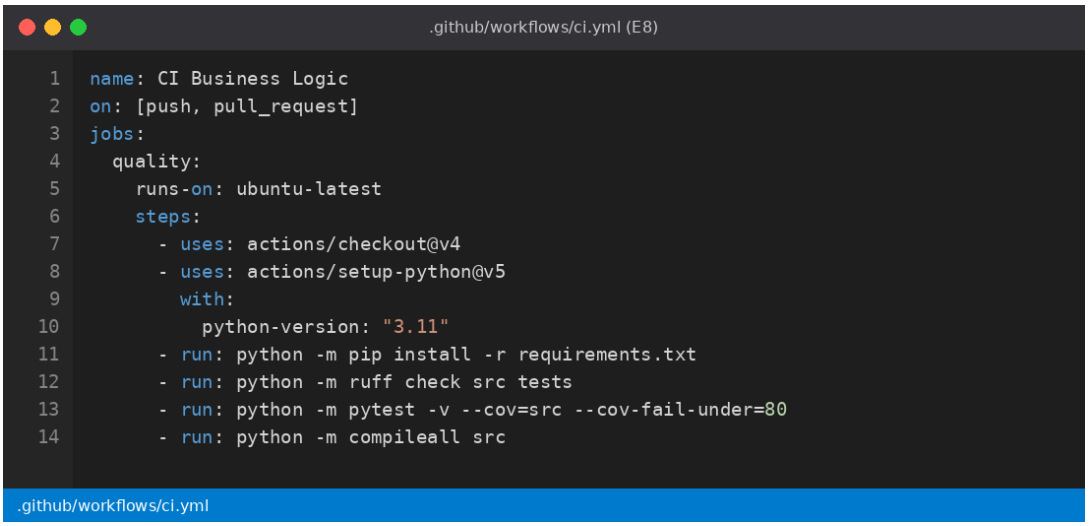


Figura 9. Workflow ci.yml: Ruff, pytest con umbral de 80 % y compilación de src/.



Figura 9b. Primera ejecución del pipeline: verde (imagen original).

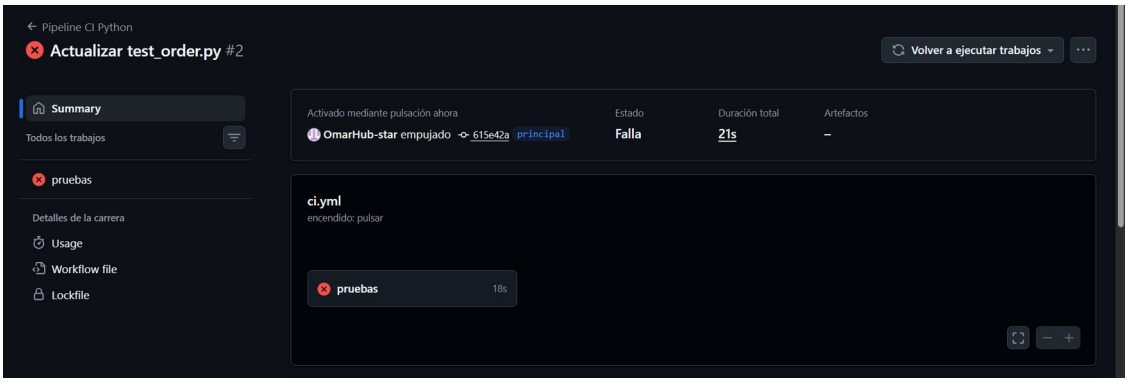


Figura 9c. Falla controlada: rojo (imagen original).

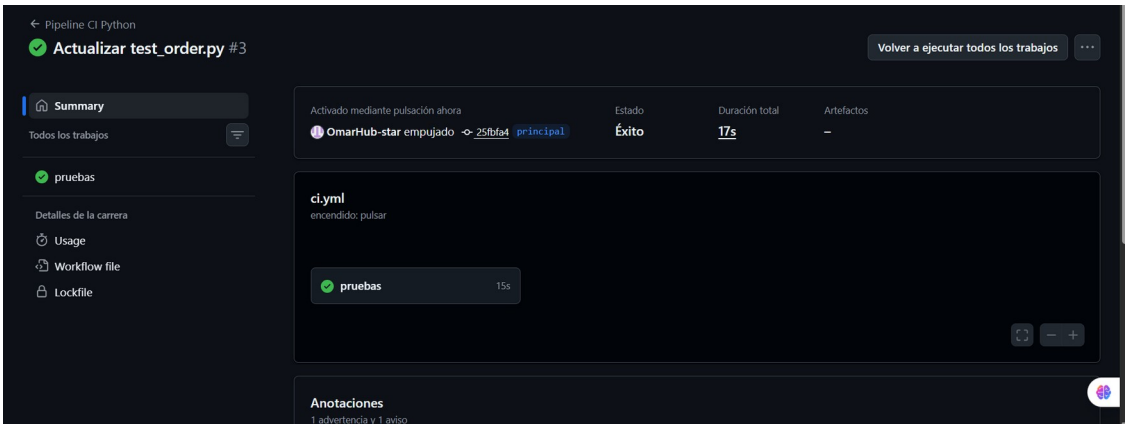


Figura 9d. Recuperación: verde (imagen original).

PASO	RESULTADO	EJECUCIÓN
Primera ejecución	Verde (éxito)	run 41965777703
Falla controlada	Rojo: aserción 100 → 999	run 41966875072
Recuperación	Verde: aserción restaurada	run 41966983525
Protección de rama	No configurada	Requiere ruleset en GitHub

Base de URL de los runs: https://github.com/OmarHub-star/Unidad-5-COMPONENTES-DE-L-GICA-DE-NEGOCIOS/actions/runs/<número>.

Observación: el workflow publicado debe incluir Ruff y compilación de src/, como pide la guía; si no los tiene, agregarlos antes de la entrega.

Aplicación al PIA: Eventos Nova (E9)

Entidades y servicios tomados de los diagramas del repositorio del equipo: Propuesta, VersionDePropuesta, LineaDeServicio, Evento, Pago, SolicitudDeProveedor; Dinero y Aforo; CotizacionService, ContratacionService, CambiosService; repositorios y UnitOfWork.

Reglas elegidas: R1 Propuesta (aceptar con versión y al menos una línea). R2 Evento (registrar anticipo mayor a cero por un responsable). R3 SolicitudDeProveedor (confirmar, rechazar y expirar).

Importante: los umbrales (anticipo cero, instante exacto de expiración) son criterios propuestos; deben confrontarse con las reglas aprobadas del cliente antes de la entrega.

ID	R	TIPO	ENTRADA	ESPERADO	ESTADO
PIA-01	R1	Éxito	Propuesta con 1 versión y 2 líneas; aceptar()	ACEPTADA	PASSED
PIA-02	R1	Error	Propuesta sin versiones; aceptar()	DominioError	PASSED
PIA-03	R1	Límite	Versión con exactamente 1 línea	ACEPTADA	PASSED
PIA-04	R2	Éxito	Anticipo 100 MXN; responsable	Pago registrado	PASSED
PIA-05	R2	Error	Anticipo -1 MXN	DominioError	PASSED
PIA-06	R2	Límite	Anticipo 0 MXN	DominioError (a confirmar)	PASSED
PIA-07	R3	Éxito	Solicitud pendiente; confirmar() antes de vencer	CONFIRMADA	PASSED
PIA-08	R3	Error	Solicitud expirada; confirmar()	DominioError	PASSED
PIA-09	R3	Límite	expirar() en el instante exacto de vencimiento	EXPIRADA (a confirmar)	PASSED

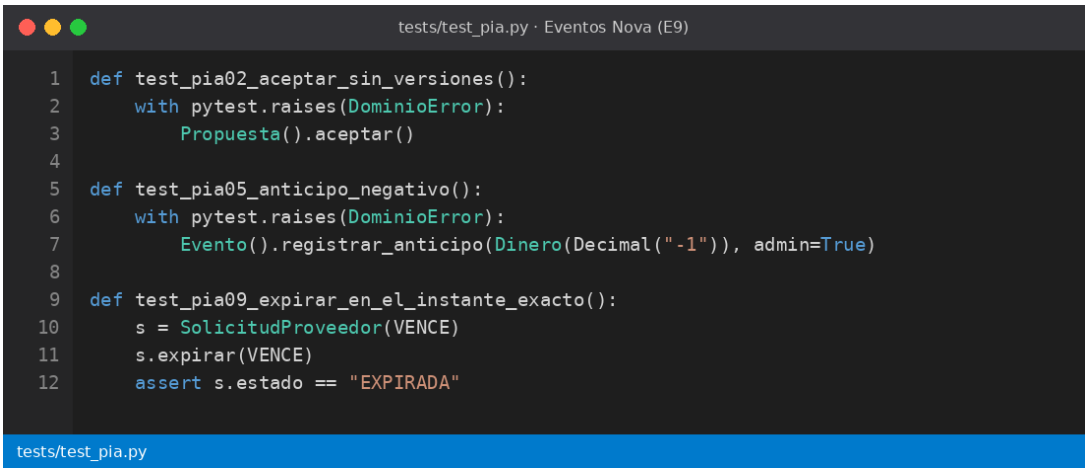


Figura 10. Fragmento de tests/test_pia.py (código completo en los archivos adjuntos).

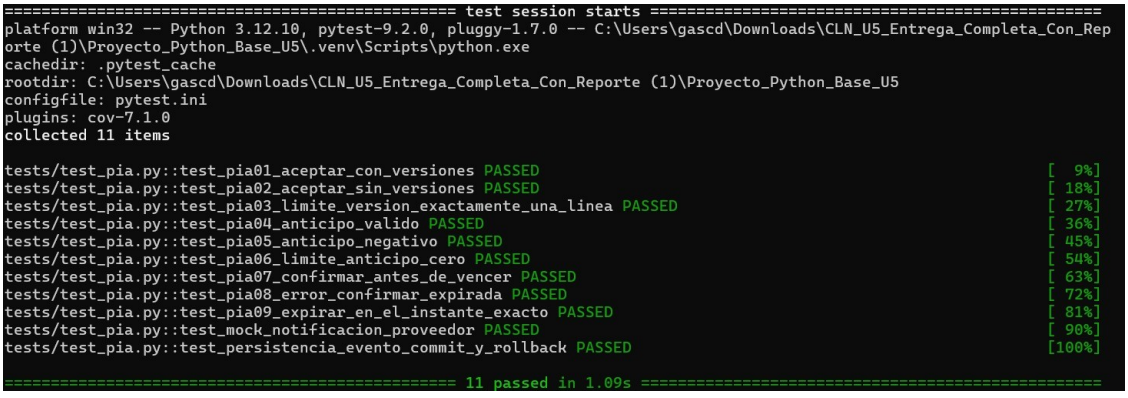


Figura 11. Evidencia E9: 11 pruebas PASSED (9 casos del PIA, mock y persistencia), imagen original.

Persistencia y mock: además de los nueve casos, se ejecutaron test_persistencia_evento_commit_y_rollback (un Evento se guarda con commit en SQLite en memoria y se recupera por id; con rollback se descarta) y

test_mock_notificacion_proveedor (adaptador ficticio de notificación a proveedores probado con Mock, sin envíos reales). Las 11 pruebas pasaron en 1.09 s (Figura 11).

Decisiones de diseño: las reglas viven en las entidades del dominio; los valores monetarios usan Dinero (Decimal y moneda); el tiempo se recibe como parámetro (ahora) para que las pruebas sean deterministas.

Estrategia E2E propuesta

Recorrido futuro: crear propuesta con versión y líneas → aceptar → registrar evento y anticipo → solicitar confirmación al proveedor → persistir con UnitOfWork → consultar estados. No se ejecuta porque el proyecto no expone API ni interfaz.

Matriz de pruebas del laboratorio (CP01–CP15)

Resumen de la matriz; el detalle va en MATRIZ_RESULTADOS.csv. Confirmar cada fila con la ejecución propia.

CASO	RESULTADO ESPERADO	ESTADO
CP01	Cobro 120.00	APROBADO
CP02	Cobro 170.00 (VIP)	APROBADO
CP03	CustomerHasDebt	APROBADO
CP04	ValueError (total 0)	APROBADO
CP05	ValueError (total negativo)	APROBADO
CP06	Cobro 850.00	APROBADO
CP07	Cobro 0.01	APROBADO
CP08	OrderAlreadyConfirmed	APROBADO
CP09	Cobro 255.00	APROBADO
CP10	Cobro 75.00	APROBADO
CP11	CustomerHasDebt	APROBADO
CP12	Cobro 84.99	APROBADO
CP13	RuntimeError: payment rejected	APROBADO
CP14	Registro visible tras commit	APROBADO
CP15	Registro ausente tras rollback	APROBADO

Uso responsable de IA

Se usó IA para proponer casos de éxito, error y límite. Caso corregido: la prueba de rechazo de pago sugerida usaba un pedido regular de 50; no verificaba el escenario VIP de 200 con cargo de 170 que pide la guía. Se reemplazó por test_gateway_rejects_vip_200_charge_170 y se comprobó con el mock la llamada exacta y la excepción. Una prueba que pasa no basta: debe coincidir con la regla aprobada.

Lista de entrega

ENTREGABLE	QUÉ DEBE DEMOSTRAR	LISTO (☐/☑️)
src/ y tests/	Pruebas reproducibles	☐
Matriz CSV	Entradas, expectativas, resultado, estado	☐

ENTREGABLE	QUÉ DEBE DEMOSTRAR	LISTO (☑□)
Cobertura HTML	≥ 80 % sobre el alcance acordado	☐
Pipeline CI	Verde, roja y recuperada	☐
README	Instalación, comandos, arquitectura	☐
Evidencias E1–E9	Capturas legibles y explicación	☐
Seguridad	Sin .venv, tokens ni credenciales	☐

Conclusiones:

El defecto de negocio más costoso en producción sería confirmar un pedido cuando el pago fue rechazado. El sistema trataría como completada una venta sin ingreso, con entregas no cobradas, conflictos con clientes y errores contables. En el proyecto, el servicio confirma el pedido antes de consultar la pasarela, así que aun con un rechazo simulado el estado queda CONFIRMED. La prueba de pago rechazado para un cliente VIP de 200 documenta que se intenta cobrar 170, que ocurre el rechazo y que el pedido permanece confirmado. Para evitarlo cambiaría el flujo: la confirmación definitiva solo después de una autorización exitosa y la persistencia dentro de una transacción controlada. Mantendría pruebas de regresión para rechazo, reintentos y doble confirmación, ejecutadas automáticamente por el pipeline de integración continua.

Se verificó que las reglas del sistema de pedidos pueden comprobarse de forma automática y reproducible: pruebas unitarias para el descuento VIP, TDD para la doble confirmación, pruebas de integración con SQLAlchemy para commit y rollback, mocks para la pasarela de pago, una matriz de autorización con el límite de 24 horas, cobertura medida con pytest-cov y un pipeline de integración continua que pasó de verde a rojo y de nuevo a verde. Lo aprendido se aplicó a nueve casos del PIA de Eventos Nova.